Платформено-независими програмни езици

Лекция 5

Generics и колекции

Обобщени типове, Java Collections Framework – списъци, множества, опашки и асоциативни масиви



проф. д-р Даниела Гоцева
Учебна 2026/2027 г.

Съдържание на лекцията

1. Обобщени типове (generics)
2. Обвиващи класове
3. Java Collections Framework
4. Списъци и опашки
5. Множества
6. Асоциативни масиви (Map)
7. Алгоритми и избор на колекция

01

Обобщени типове (generics)

Типово безопасен код за много типове

Проблемът без generics

- Преди Java 5 колекциите съхраняваха `Object`
- При четене – задължително **преобразуване на типа**
- Грешен тип се открива едва **при изпълнение** – `ClassCastException`
- Generics преместват проверката в **компилатора**
- Код, написан веднъж, работи за много типове – и е безопасен

Без и с generics

Без generics (до Java 5)

- `List list = new ArrayList();`
- `list.add("текст");`
- `list.add(42); // позволено!`
- `String s = (String) list.get(1);`
- → `ClassCastException` при изпълнение

С generics

- `List<String> list = new ArrayList<>();`
- `list.add("текст");`
- `list.add(42); // грешка при компилиране`
- `String s = list.get(0);`
- без преобразуване, без изненади

Обобщен клас

- Типов параметър в < >: T, E, K, V
- Използва се като обикновен тип в класа
- При създаване се задава конкретен тип
- <> (diamond) – компилаторът извежда типа

```
public class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }

    public void set(T value) {
        this.value = value;
    }
}

Box<String> b = new Box<>("Java");
String s = b.get();        // без cast
Box<Integer> n = new Box<>(42);
```

Няколко типови параметъра

- Класът може да има няколко параметъра
- С `record` – още по-кратко
- Конвенции: `T` – тип, `E` – елемент, `K` – ключ, `V` – стойност, `R` – резултат

```
public record Pair<A, B>(A first, B second) {

    public <C> Pair<A, C> withSecond(C c) {
        return new Pair<>(first, c);
    }
}

Pair<String, Integer> p =
        new Pair<>("Ана", 6);

String name = p.first();
int grade = p.second();

var q = p.withSecond(5.75);
// Pair<String, Double>
```

Обобщени методи

- Типовият параметър се декларира **преди** типа на резултата
- Компилаторът извежда типа от аргументите
- Може да е и в необобщен клас

```
public class Utils {

    public static <T> T first(List<T> list) {
        return list.isEmpty() ? null : list.get(0);
    }

    public static <T> void swap(T[] a,
                                int i, int j) {
        T tmp = a[i];
        a[i] = a[j];
        a[j] = tmp;
    }
}

String s = Utils.first(List.of("a", "b"));
Integer[] arr = {1, 2, 3};
Utils.swap(arr, 0, 2);   // [3, 2, 1]
```

Пример: обобщен стек

```
public class Stack<T> implements Iterable<T> {
    private final List<T> items = new ArrayList<>();

    public void push(T item) { items.add(item); }
    public T pop() {
        if (items.isEmpty()) throw new NoSuchElementException("Празен стек");
        return items.remove(items.size() - 1);
    }

    public T peek()          { return items.get(items.size() - 1); }
    public boolean isEmpty() { return items.isEmpty(); }

    @Override public Iterator<T> iterator() { return items.iterator(); }
}

Stack<String> s = new Stack<>();
s.push("a"); s.push("b");
String top = s.pop();      // "b" — без преобразуване на типа
```

Ограничени типове (bounded types)

- `<T extends Тип>` - T трябва да е наследник или да имплементира `Тип`
- Позволява да извикваме методите на `Тип`
- Няколко ограничения: `<T extends A & B>`

```
public static <T extends Comparable<T>>
        T max(List<T> list) {
    T best = list.get(0);
    for (T x : list) {
        if (x.compareTo(best) > 0) {
            best = x;
        }
    }
    return best;
}

max(List.of(3, 9, 4));           // 9
max(List.of("круша", "ябълка")); // ябълка
```

Wildcards (заместители)

- `List<Integer>` **не е** наследник на `List<Number>`!
- `?` – неизвестен тип
- `? extends T` – T или наследник: **четем** елементи като T
- `? super T` – T или родител: **записваме** елементи от тип T
- Правило **PECS**: *Producer Extends, Consumer Super*
  - източник на данни → `extends`
  - приемник на данни → `super`

Wildcards – пример

```java
// Чете числа – "производител" -> extends
static double sum(List<? extends Number> list) {
    double s = 0;
    for (Number n : list) s += n.doubleValue();
    return s;
}

// Записва цели числа – "консуматор" -> super
static void fill(List<? super Integer> list, int count) {
    for (int i = 1; i <= count; i++) list.add(i);
}

sum(List.of(1, 2, 3));          // List<Integer> – OK
sum(List.of(1.5, 2.5));         // List<Double>  – OK

List<Number> nums = new ArrayList<>();
fill(nums, 5);                  // List<Number>  – OK
```

Изтриване на типовете (type erasure)

- Generics съществуват **само по време на компилиране**
- В байткода `List<String>` и `List<Integer>` са просто `List`
- Следствия – **не може**:
  - `new T()` или `new T[10]`
  - `x instanceof List<String>`
  - generic параметър от примитивен тип: `List<int>`
  - статично поле от тип `T`
- Причина: съвместимост със стария код преди Java 5

02

Обвиващи класове

Примитивни типове в колекциите

Обвиващи (wrapper) класове

Примитив	Обвиващ клас	Полезни методи
`int`	`Integer`	`parseInt`, `valueOf`, `MAX_VALUE`, `compare`
`long`	`Long`	`parseLong`, `sum`
`double`	`Double`	`parseDouble`, `isNaN`, `compare`
`boolean`	`Boolean`	`parseBoolean`
`char`	`Character`	`isDigit`, `isLetter`, `toUpperCase`
`byte`, `short`, `float`	`Byte`, `Short`, `Float`	аналогични

Autoboxing и unboxing

- **Autoboxing** – автоматично `int` → `Integer`
- **Unboxing** – автоматично `Integer` → `int`
- Удобно, но има капани:
  - `null` при unboxing → `NullPointerException`
  - `==` между `Integer` сравнява референции
  - излишни обекти в цикли

```
List<Integer> list = new ArrayList<>();
list.add(5);                // autoboxing
int x = list.get(0);        // unboxing

Integer a = 1000, b = 1000;
System.out.println(a == b);         // false!
System.out.println(a.equals(b));  // true

Integer missing = null;
// int y = missing;  -> NullPointerException
```

03

Java Collections Framework

Интерфейси, реализации и алгоритми

Какво е Collections Framework?

- Единна архитектура за съхранение и обработка на групи от обекти
- **Интерфейси** – абстрактни типове: `List`, `Set`, `Queue`, `Map`
- **Реализации** – конкретни класове: `ArrayList`, `HashSet`, `HashMap`
- **Алгоритми** – сортиране, търсене, разбъркване: класът `Collections`
- Предимства: по-малко код, по-висока производителност, съвместимост между библиотеки
- Аналог на `list`, `set`, `dict` в Python – но с повече избор

Колекциите в Python и в Java

Python	Java – интерфейс	Java – типична реализация
`list`	`List<E>`	`ArrayList<E>`
`tuple`	`List.of(...)` или `record`	неизменяем списък / запис
`set`	`Set<E>`	`HashSet<E>`
`dict`	`Map<K,V>`	`HashMap<K,V>`
`collections.deque`	`Deque<E>`	`ArrayDeque<E>`
`heapq`	`Queue<E>`	`PriorityQueue<E>`
`sorted(d)` върху ключове	`SortedMap<K,V>`	`TreeMap<K,V>`

Основни интерфейси и реализации

Интерфейс	Характеристика	Основни реализации
`List<E>`	наредена, с индекси, позволява повторения	`ArrayList`, `LinkedList`
`Set<E>`	без повторения	`HashSet`, `LinkedHashSet`, `TreeSet`
`Queue<E>` / `Deque<E>`	опашка / двустранна опашка	`ArrayDeque`, `PriorityQueue`
`Map<K,V>`	двойки ключ → стойност, уникални ключове	`HashMap`, `LinkedHashMap`, `TreeMap`

Йерархия на интерфейсите

- `Iterable<E>` – всичко, което може да се обхожда с `for-each`
  - `Collection<E>` – обща група от елементи
    - `SequencedCollection<E>` (Java 21) – с определен ред: първи / последен
      - `List<E>`, `Deque<E>`
    - `Set<E>` → `SortedSet<E>` → `NavigableSet<E>`
    - `Queue<E>` → `Deque<E>`
- `Map<K,V>` – **отделна** йерархия, не е `Collection`
  - `SortedMap<K,V>` → `NavigableMap<K,V>`

Методи на интерфейса Collection

Метод	Действие
`add(e)`, `addAll(c)`	добавя елемент / всички елементи
`remove(o)`, `removeIf(p)`	премахва елемент / по условие
`contains(o)`	проверява дали съдържа (чрез `equals`)
`size()`, `isEmpty()`	брой елементи, проверка за празнота
`clear()`	изтрива всичко
`iterator()`	итератор за обхождане
`stream()`	поток за обработка (следващата лекция)
`toArray()`	превръща в масив

Обхождане и итератори

- `for-each` – най-простият начин
- `Iterator` – позволява **безопасно премахване** по време на обхождане
- `forEach(ламбда)` – функционален стил
- Промяна на колекцията в `for-each` → `ConcurrentModificationException`

```
List<String> names = new ArrayList<>(
        List.of("Ана", "Боян", "Ив", "Вера"));

for (String n : names) {
    System.out.println(n);
}

Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().length() < 3) {
        it.remove();           // безопасно
    }
}

names.removeIf(n -> n.startsWith("Б"));
names.forEach(System.out::println);
```

04

Списъци и опашки

List, ArrayList, LinkedList, Deque, PriorityQueue

ArrayList

- Динамичен масив – расте автоматично
- Бърз достъп по индекс – O(1)
- Добавяне в края – O(1) средно
- Вмъкване/изтриване в началото – O(n)
- **Най-често използваният** списък

```java
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
list.add(0, "C");          // вмъкване
list.set(2, "Kotlin");     // замяна

System.out.println(list);           // [C, Java, Kotlin]
System.out.println(list.get(1));    // Java
System.out.println(list.indexOf("C")); // 0
System.out.println(list.size());    // 3

list.remove("C");
list.remove(0);            // по индекс
```

ArrayList или LinkedList?

Операция	`ArrayList`	`LinkedList`
достъп по индекс `get(i)`	O(1)	O(n)
добавяне в края	O(1) средно	O(1)
добавяне/изтриване в началото	O(n)	O(1)
изтриване по време на обхождане с итератор	O(n)	O(1)
памет	компактен масив	възел + 2 указателя за всеки елемент
препоръка	почти винаги	рядко – за опашки по-добре `ArrayDeque`

Неизменяеми списъци

- `List.of(...)` – неизменяем списък (Java 9)
- `List.copyOf(c)` – неизменяемо копие
- Опит за промяна → `UnsupportedOperationException`
- `Arrays.asList` – фиксиран размер, но позволява `set`

```
List<String> days =
        List.of("пон", "вт", "ср");
// days.add("чет");
//   -> UnsupportedOperationException

List<String> copy =
        new ArrayList<>(days);   // изменяемо
copy.add("чет");

List<Integer> fixed =
        Arrays.asList(1, 2, 3);
fixed.set(0, 10);    // OK
// fixed.add(4);     // грешка
```

Подсписъци и ListIterator

- `subList(from, to)` – **изглед** към част от списъка
- Промяна на изгледа променя и оригинала
- `ListIterator` – обхождане в двете посоки и замяна на елементи

```
List<Integer> nums = new ArrayList<>(
        List.of(1, 2, 3, 4, 5, 6));

List<Integer> mid = nums.subList(2, 4);
// [3, 4]
mid.clear();
System.out.println(nums);   // [1, 2, 5, 6]

ListIterator<Integer> it = nums.listIterator();
while (it.hasNext()) {
    int x = it.next();
    if (x % 2 == 0) it.set(x * 10);
}
System.out.println(nums);   // [1, 20, 5, 60]
```

Сортиране на списъци

- `list.sort(null)` или `Collections.sort` – естествена наредба (`Comparable`)
- `list.sort(comparator)` – по зададен критерий
- Сортирането е **стабилно** – равните запазват реда си

```
List<String> words = new ArrayList<>(
        List.of("круша", "Ябълка", "банан"));

Collections.sort(words);
// [Ябълка, банан, круша] – главните първо

words.sort(String.CASE_INSENSITIVE_ORDER);
// [банан, круша, Ябълка]

words.sort(Comparator
        .comparing(String::length)
        .thenComparing(Comparator.reverseOrder()));
// [круша, банан, Ябълка]
```

SequencedCollection (Java 21)

- Общ интерфейс за колекции с **определен ред**
- Единни методи за първия и последния елемент
- `reversed()` – изглед в обратен ред
- Преди: `list.get(list.size() - 1)`

```
List<String> list = new ArrayList<>(
        List.of("a", "b", "c"));

list.getFirst();       // "a"
list.getLast();        // "c"
list.addFirst("z");    // [z, a, b, c]
list.removeLast();     // [z, a, b]

for (String s : list.reversed()) {
    System.out.print(s);   // baz
}
```

Опашки и стекове: Deque

- `Deque` – двустранна опашка: добавяне и вземане от **двата края**
- `ArrayDeque` – препоръчителната реализация
- Като **опашка** (FIFO): `offer` / `poll`
- Като **стек** (LIFO): `push` / `pop`
- Класът `Stack` е остарял – не го използвайте

```
Deque<String> queue = new ArrayDeque<>();
queue.offer("клиент 1");
queue.offer("клиент 2");
queue.poll();              // "клиент 1"

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
stack.push(3);
stack.pop();               // 3
stack.peek();              // 2
```

Пример: проверка на скоби със стек

- Отваряща скоба → в стека
- Затваряща → трябва да съвпада с върха на стека
- Накрая стекът трябва да е празен

```
static boolean balanced(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(', '[', '{' -> st.push(c);
            case ')' -> { if (st.isEmpty()
                    || st.pop() != '(') return false; }
            case ']' -> { if (st.isEmpty()
                    || st.pop() != '[') return false; }
            case '}' -> { if (st.isEmpty()
                    || st.pop() != '{') return false; }
            default -> { }
        }
    }
    return st.isEmpty();
}
```

Приоритетна опашка

- `PriorityQueue` – винаги връща **най-малкия** елемент
- Наредба – естествена или чрез `Comparator`
- Реализирана с двоична пирамида (heap): O(log n)
- Приложения: планиране на задачи, алгоритъм на Дейкстра

```
record Task(String name, int priority) { }

PriorityQueue<Task> pq = new PriorityQueue<>(
        Comparator.comparingInt(Task::priority));

pq.add(new Task("отговор на имейл", 3));
pq.add(new Task("авария на сървъра", 1));
pq.add(new Task("кафе", 5));

while (!pq.isEmpty()) {
    System.out.println(pq.poll().name());
}
// авария на сървъра, отговор на имейл, кафе
```

05

Множества

HashSet, LinkedHashSet, TreeSet

Реализации на Set

Клас	Ред на елементите	Сложност	Изисква
`HashSet`	без гарантиран ред	O(1)	`equals` + `hashCode`
`LinkedHashSet`	ред на добавяне	O(1)	`equals` + `hashCode`
`TreeSet`	сортиран	O(log n)	`Comparable` или `Comparator`

HashSet и LinkedHashSet

- Повторенията се игнорират
- `add` връща `false`, ако елементът вече е там
- Бърза проверка за принадлежност

```
Set<String> tags = new HashSet<>();
tags.add("java");
tags.add("oop");
boolean added = tags.add("java");  // false
System.out.println(tags.size());   // 2
System.out.println(tags.contains("oop"));

// Премахване на повторения, запазвайки реда
List<Integer> nums = List.of(3, 1, 3, 2, 1);
Set<Integer> unique =
        new LinkedHashSet<>(nums);
System.out.println(unique);   // [3, 1, 2]
```

TreeSet – сортирано множество

- Елементите винаги са подредени
- Методи за навигация: `first`, `last`, `floor`, `ceiling`, `headSet`, `tailSet`
- Подходящо за диапазонни заявки

```
TreeSet<Integer> scores = new TreeSet<>(
        List.of(72, 95, 60, 88, 41));

System.out.println(scores);
// [41, 60, 72, 88, 95]
scores.first();           // 41
scores.last();            // 95
scores.ceiling(70);       // 72 – най-малкият >= 70
scores.floor(70);         // 60 – най-големият <= 70
scores.headSet(72);       // [41, 60]
scores.descendingSet();   // [95, 88, 72, 60, 41]
```

Операции с множества

- **Обединение** – `addAll`
- **Сечение** – `retainAll`
- **Разлика** – `removeAll`
- Работим върху копия, за да запазим оригиналите

```
Set<String> java = Set.of("Ана", "Боян", "Вера");
Set<String> python = Set.of("Боян", "Гошо");

Set<String> union = new TreeSet<>(java);
union.addAll(python);
// [Ана, Боян, Вера, Гошо]

Set<String> both = new TreeSet<>(java);
both.retainAll(python);
// [Боян]

Set<String> onlyJava = new TreeSet<>(java);
onlyJava.removeAll(python);
// [Ана, Вера]
```

Записи като елементи на множество

- `record` генерира `equals` и `hashCode` по всички компоненти
- Два записа с еднакви данни са **равни**
- С обикновен клас без `equals`/`hashCode` – два „еднакви“ обекта в множеството

```
record Seat(int row, int number) { }

Set<Seat> taken = new HashSet<>();
taken.add(new Seat(5, 12));
taken.add(new Seat(5, 12));   // дубликат

System.out.println(taken.size());   // 1
System.out.println(taken.contains(
        new Seat(5, 12)));          // true
```

EnumSet и EnumMap

- Специализирани колекции за ключове от `enum`
- Вътрешно – битова маска / масив: много бързи
- Подредени по реда на константите

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumSet<Day> weekend =
        EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> work =
        EnumSet.complementOf(weekend);

EnumMap<Day, Integer> hours =
        new EnumMap<>(Day.class);
hours.put(Day.MON, 6);
hours.put(Day.WED, 3);
System.out.println(hours);  // {MON=6, WED=3}
```



Обекти в HashSet и HashMap изискват коректни equals() и hashCode()

Иначе „еднакви“ обекти се третират като различни. Записите (`record`) решават проблема автоматично. За `TreeSet`/`TreeMap` е нужна наредба – `Comparable` или `Comparator`.

06

Асоциативни масиви (Map)

Двойки ключ → стойност

Реализации на Map

Клас	Ред на ключовете	Сложност	Типична употреба
`HashMap`	без гарантиран ред	O(1)	по подразбиране
`LinkedHashMap`	ред на добавяне (или на достъп)	O(1)	запазване на реда, кеш (LRU)
`TreeMap`	сортиран по ключ	O(log n)	сортирани отчети, диапазони

Основни операции с Map

- `put(k, v)` – добавя или заменя
- `get(k)` – стойност или `null`
- `getOrDefault(k, d)` – стойност по подразбиране
- `containsKey`, `remove`, `size`
- Подобно на `dict` в Python

```
Map<String, Integer> phone = new HashMap<>();
phone.put("Ана", 888123456);
phone.put("Боян", 877654321);
phone.put("Ана", 899000111);  // заменя

phone.get("Ана");             // 899000111
phone.get("Вера");            // null
phone.getOrDefault("Вера", 0);  // 0
phone.containsKey("Боян");  // true
phone.remove("Боян");

Map<String, Integer> fixed =
        Map.of("a", 1, "b", 2);  // неизменяем
```

Обхождане на Map

- `keySet()` – множеството на ключовете
- `values()` – колекция от стойностите
- `entrySet()` – двойки `Map.Entry<K,V>`
- `forEach((k, v) -> ...)` – най-кратко

```
Map<String, Double> avg = new TreeMap<>();
avg.put("Вера", 5.20);
avg.put("Ана", 5.75);
avg.put("Боян", 4.50);

for (Map.Entry<String, Double> e
        : avg.entrySet()) {
    System.out.println(e.getKey()
            + " -> " + e.getValue());
}

avg.forEach((name, g) ->
        System.out.printf("%s: %.2f%n", name, g));

for (String name : avg.keySet()) { /*...*/ }
```

Удобни методи: merge и computeIfAbsent

- `putIfAbsent` – добавя само ако ключът липсва
- `merge` – комбинира стара и нова стойност
- `computeIfAbsent` – създава стойност при нужда
- Заменят много `if (map.containsKey(...))`

```
Map<String, Integer> count = new HashMap<>();
count.merge("java", 1, Integer::sum);
count.merge("java", 1, Integer::sum);
// {java=2}

Map<String, List<String>> groups =
        new HashMap<>();
groups.computeIfAbsent("2 курс",
        k -> new ArrayList<>()).add("Ана");
groups.computeIfAbsent("2 курс",
        k -> new ArrayList<>()).add("Боян");
// {2 курс=[Ана, Боян]}
```

Собствени обекти като ключове

- Ключът трябва да има коректни `equals` и `hashCode`
- Ключът трябва да е **неизменяем** – иначе се „губи“ в таблицата
- `record` е идеален ключ

```
record Cell(int row, int col) { }

Map<Cell, String> board = new HashMap<>();
board.put(new Cell(0, 0), "X");
board.put(new Cell(1, 1), "O");

board.get(new Cell(1, 1));        // "O"
board.containsKey(new Cell(2, 2)); // false

for (var e : board.entrySet()) {
    Cell c = e.getKey();
    System.out.println(c.row() + "," + c.col()
            + " = " + e.getValue());
}
```

Пример: честота на думите

```
String text = """
        Java е език. Java е платформа.
        Python е език за първи курс.
        """;

Map<String, Integer> freq = new TreeMap<>();
for (String w : text.toLowerCase().split("[^\\p{L}]+")) {
    if (!w.isBlank()) {
        freq.merge(w, 1, Integer::sum);
    }
}

freq.forEach((w, n) -> System.out.println(w + ": " + n));
// език: 2   за: 1   java: 2   е: 3   ...
```

`TreeMap` подрежда думите; `\p{L}` – всяка буква от Unicode, вкл. кирилица

Пример: групиране на студенти по курс

```
record Student(String name, int course) { }

List<Student> all = List.of(
        new Student("Ана", 2), new Student("Боян", 1),
        new Student("Вера", 2), new Student("Гошо", 3));

Map<Integer, List<String>> byCourse = new TreeMap<>();
for (Student s : all) {
    byCourse.computeIfAbsent(s.course(), k -> new ArrayList<>())
            .add(s.name());
}

System.out.println(byCourse);
// {1=[Боян], 2=[Ана, Вера], 3=[Гошо]}
```

В следващата лекция ще направим същото на един ред със `Collectors.groupingBy`

TreeMap – навигация по ключове

- Ключовете са сортирани
- `firstKey`, `lastKey`, `floorKey`, `ceilingKey`
- `headMap`, `tailMap`, `subMap` – диапазони
- Пример: оценка по точки

```
TreeMap<Integer, String> scale =
        new TreeMap<>(Map.of(
             0, "Слаб 2",
             50, "Среден 3",
             65, "Добър 4",
             80, "Мн. добър 5",
             90, "Отличен 6"));

static String grade(int points) {
    return scale.floorEntry(points)
                .getValue();
}

grade(72);   // Добър 4
grade(95);   // Отличен 6
```

Пример: телефонен указател

- `TreeMap` – азбучен ред на имената
- `subMap` – всички имена от „Б“ до „Г“
- `Map<String, List<String>>` – няколко номера за едно име

```
TreeMap<String, List<String>> book =
        new TreeMap<>();

static void add(String name, String phone) {
    book.computeIfAbsent(name,
            k -> new ArrayList<>()).add(phone);
}

add("Боян", "0888 111 222");
add("Ана", "0877 333 444");
add("Ана", "02 555 6666");
add("Георги", "0899 777 888");

System.out.println(book.subMap("Б", "Г"));
// {Боян=[0888 111 222]}
book.firstKey();    // Ана
```

07

Алгоритми и избор на колекция

Класът Collections и добри практики

Класът Collections

Метод	Действие
`sort(list)`, `sort(list, cmp)`	сортира списък
`reverse(list)`, `shuffle(list)`	обръща / разбърква
`max(c)`, `min(c)`	най-голям / най-малък елемент
`frequency(c, o)`	колко пъти се среща елемент
`binarySearch(list, key)`	двоично търсене в сортиран списък
`unmodifiableList(list)`	изглед само за четене
`nCopies(n, o)`, `emptyList()`	помощни колекции

Коя колекция да изберем?

Нужда	Избор
списък, достъп по индекс	`ArrayList`
уникални елементи, бърза проверка	`HashSet`
уникални и сортирани	`TreeSet`
ключ → стойност	`HashMap`
ключ → стойност, сортирани ключове	`TreeMap`
опашка или стек	`ArrayDeque`
най-приоритетният елемент	`PriorityQueue`
запазване на реда на добавяне	`LinkedHashSet`, `LinkedHashMap`

Сложност на основните операции

Колекция	добавяне	търсене / `contains`	изтриване	достъп по индекс/ключ
`ArrayList`	O(1)*	O(n)	O(n)	O(1)
`LinkedList`	O(1)	O(n)	O(1)**	O(n)
`HashSet` / `HashMap`	O(1)	O(1)	O(1)	O(1)
`TreeSet` / `TreeMap`	O(log n)	O(log n)	O(log n)	O(log n)
`ArrayDeque`	O(1)	O(n)	O(1) в краищата	–
`PriorityQueue`	O(log n)	O(n)	O(log n)	–

Добри практики

- Декларирайте променливите с типа на **интерфейса**: `List<String> x = new ArrayList<>();`
- Използвайте `<>` (diamond) и `var`, когато типът е ясен
- Връщайте празна колекция, а не `null`
- Предпочитайте неизменяеми колекции: `List.of`, `List.copyOf`
- Не променяйте колекция, докато я обхождате с `for-each`
- Ключовете в `HashMap` трябва да са **неизменяеми**
- Не използвайте „сурови“ типове (`List` без `< >`)

Колекции и нишки

- Обикновените колекции (`ArrayList`, `HashMap`) **не са** безопасни при едновременна промяна от няколко нишки
- Възможни последици: загубени елементи, повредена структура, изключения
- Решения:
  - неизменяеми колекции (`List.of`) – безопасни за четене
  - конкурентни колекции: `ConcurrentHashMap`, `CopyOnWriteArrayList`
  - синхронизация
- Подробно – в лекцията за нишки

Обобщение

- Generics – типова безопасност при компилиране; `<T>`, `<T extends X>`, `? extends`, `? super`
- Type erasure – generics не съществуват при изпълнение
- Обвиващи класове и autoboxing – примитиви в колекциите
- `List` – ред и индекси; `ArrayList` по подразбиране
- `Set` – уникалност; `HashSet`, `LinkedHashSet`, `TreeSet`
- `Deque` – опашка и стек; `PriorityQueue` – по приоритет
- `Map` – ключ → стойност; `merge`, `computeIfAbsent`, `getOrDefault`
- Коректни `equals`/`hashCode` или `Comparable` за елементите

Въпроси за самопроверка

- Какви проблеми решават generics?
- Защо `List<Integer>` не е подтип на `List<Number>`?
- Обяснете правилото PECS с пример.
- Какво ще отпечата `Integer a = 1000, b = 1000; a == b`?
- Кога бихте използвали `LinkedHashSet` вместо `HashSet`?
- Как да преброим срещанията на думите в текст с `Map`?
- Защо стекът в Java се реализира с `ArrayDeque`, а не със `Stack`?

Въпроси?

Следваща лекция: Stream API и функционален стил

